

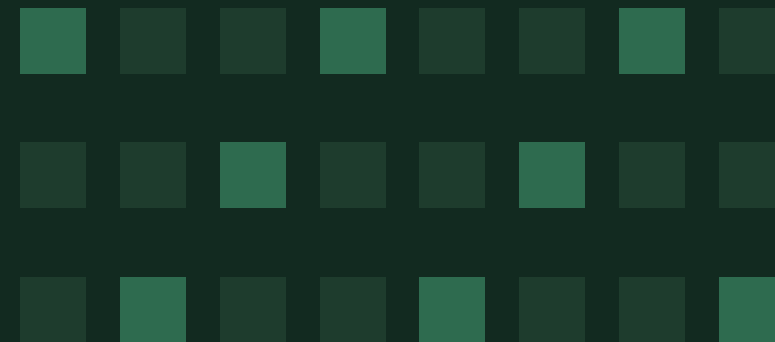
ENTREGA FINAL · UNIDADE IV · PUC MINAS · 2026/2

Rastreabilidade do gado de corte por eventos

Do cadastro do animal ao embarque no frigorífico — quatro serviços conversando só por evento.

Equipe 04

Paulo Cidrão Gomes Torres · João Almeida Barbosa Júnior · João Pedro Schlindwein · João Pedro Correia Barros · Matheus Chaves Ferreira · Rafael Corrêa Zart



Que problema existe na fazenda?

Antes da tecnologia — o que fazia esse sistema ser necessário.

A PERGUNTA QUE O SISTEMA RESPONDE

“O que é verdade sobre este animal agora — e dá para provar que aconteceu na ordem em que aconteceu?”

Formar um lote

exige peso e idade atuais de cada animal candidato

Aprovar uma venda

exige que o peso atinja a meta do contrato com o frigorífico

Aceitar um embarque

exige vacinação em dia — o frigorífico recusa na triagem

Três decisões de dinheiro dependem da mesma coisa: um registro único e ordenado do que aconteceu com cada animal — que antes vivia em planilha e telefonema.

Quatro serviços, seis eventos

A única via entre eles é o tópico — nenhum chama o outro.

PUBLICAM



Duas chaves de partição, não uma

animalId nos fatos do animal, loteId nos fatos do lote — a ordem só importa dentro de cada um.

Um group.id por pergunta

Grupos diferentes no mesmo tópico não disputam partição — cada um lê o fluxo inteiro.

O que trafega no fio

Envelope CloudEvents 1.0 em modo binário.

CABECALHOS · QUEM ROTEIA LE SO ISTO

```
ce_id      : evt-vac-2026-000123
ce_type    : gado.animal.vacinacao-registrada.v1
ce_source  : /fazenda-corte/vacinacao-service
ce_time    : 2026-08-20T09:15:00Z
ce_subject : animal/AN-004821
```

CORPO · SO O FATO DE NEGOCIO

```
{
  "eventoId": "evt-vac-2026-000123",
  "animalId": "AN-004821",
  "vacina": "Febre Aftosa",
  "validade": "2027-02-20T23:59:59Z"
}
```

O eventold é o que torna o consumidor idempotente

Confirmamos o offset só depois do commit — at-least-once. Então o mesmo evento pode chegar duas vezes, e chega. O eventold entra numa tabela com PRIMARY KEY, na MESMA transação do efeito: a segunda entrega bate na chave e é descartada.

Event sourcing da localização do lote

ADR-005 · o agregado Lote, no recorte de onde ele está.

O PROBLEMA

“Em qual pasto está o lote agora?” não tinha resposta em lugar nenhum — nem tabela, nem coluna. Só relendo o tópico do começo, na mão.

O caminho fácil, recusado

Uma coluna `pasto_atual_id` na tabela `lote`, atualizada por `UPDATE` a cada movimentação. Duas linhas de código.

Mas o `UPDATE` apaga por onde o lote passou — e se a coluna corromper, não há de onde tirar o valor certo.

O que fizemos

Event store append-only com PK (`lote_id`, `versao`) — a própria numeração do stream detecta escrita concorrente.

A localização atual vira projeção descartável: apaga e reconstrói dobrando os eventos.

A CONSEQUÊNCIA QUE ACEITAMOS

Ler a localização deixou de ser instantâneo. A projeção é assíncrona — quem consultar durante a janela vê o pasto anterior. Toleramos alguns minutos, registrado por tela, porque nenhuma regra automática reage a essa consulta hoje.

A mensagem que nunca vai processar

O caminho inteiro — do retry à DLQ, e de volta.



Só depois de gravar é que o offset é confirmado — a partição não trava, e nada some em silêncio.

Dois modos de falha, um caminho

Poison message que não desserializa: grava os bytes originais do fio.
Regra de negócio que sempre lança: grava o evento mais o envelope CloudEvents inteiro e a posição Kafka.

Reprocessar é humano, de propósito

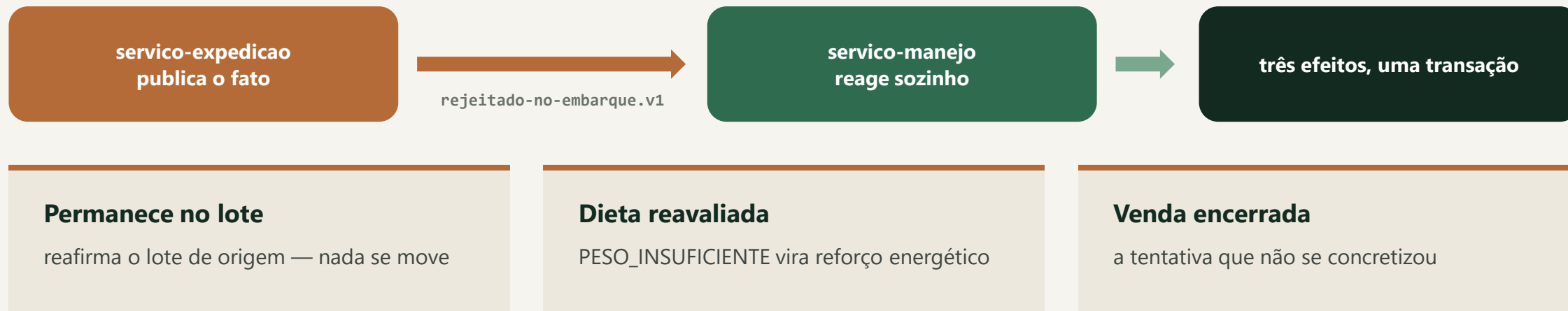
O reenvio devolve o payload ao tópico original com os cabeçalhos restaurados — sem caminho de código paralelo.
Reprocessar duas vezes não duplica efeito: a idempotência por eventoid já protege.

O CAMINHO DE VOLTA

POST /api/dlq/{id}/reprocessar — reenvia ao tópico original, cabeçalhos restaurados. Cada tentativa vira linha nova; a original fica intacta.

O frigorífico recusou o animal

Saga coreografada — nenhum componente central coordena.



NENHUM UPDATE CORRIGE O PASSADO

A recusa aconteceu e continua registrada. A compensação é o que o manejo FAZ a partir dela — um novo estado vigente, não uma reversão. E se a própria compensação falhar, cai na mesma DLQ: a saga fica incompleta e visível.

O que faríamos diferente

Recomeçando hoje — não o que deu certo.

Teríamos escrito os contratos antes do código

Publicamos eventos e só depois escrevemos o contrato. Foi ao documentar o campo validade que vimos que ele era `java.util.Date` — tipo que serializa fora do padrão dos demais campos. O evento já estava no ar assim.

Teríamos tratado `group.id` como decisão, não detalhe

Um listener novo nasceu com o `group.id` de outro e passou a disputar partições do consumidor que já funcionava. Um caractere. Hoje um `group.id` novo entra com a pergunta de negócio que ele responde escrita junto.

Teríamos medido a defasagem, não só tolerado

Documentamos que a projeção pode atrasar alguns minutos, mas não há nada que avise quando ela atrasa. A tolerância está escrita; o alarme, não. Hoje só descobre quem for procurar.